1. 用 behavioral 的 out 會要 reg

2. clock gating 的 zero_out 和 en_latched 是用 c_reg

3. case 加 default

4. assign 左邊只能 wire，右 reg、wire 都可。

   always 中，左只能放 reg，右 reg、wire 都可。

5. paremeter 一定要有初始值

6. a[10 +: 20] ⇒ a[29:10] → 共20個

   a[10 -: 3] ⇒ a[10:8] → 共3個

7. <位元>'<base><value>

   ↖ 不能是 parameter

8. and 後面可有多個 and(y, a, b, c);

9. and, or, buf if1 可不加 instance name，
   自己的 FA 一定要加 instance name，

10. 減法一定 A-B = A+(~B+1)，如何解讀結果就看是否為 signed

11. 運算時有 unsigned，結果就是 unsigned

    sign 切片連接時會變 unsigned

12. >>> 的行为和 output 取决于 input

s >>> n = s
(signed)

u >>> n = u
(unsigned)

A-1 Different Modeling Methods — 16-bit Adder

Original weight: 15%

A → B → C｜Verilog-2001/2005｜題目與作答用紙

Name: _____ Date: _____

Use different modeling methods (dataflow, structural, behavioral) to describe a 16-bit adder with 16-bit inputs named `a`, `b`, 1-bit carry-input named `ci`, 16-bit output `s`, and 1-bit carry-out `co`. Use parameter `n` for the bit-width of `a`, `b`, `s` to make the code more flexible. Write a complete Verilog code.

1.1 Use dataflow modeling method to describe the adder.

1.2 Use structural modeling method to describe the adder assuming full-adder module named `FA(cout, sum, x, y, cin)` is available.

假設

1.3 Use behavioral modeling method to describe the adder.

ANSWER / 作答區 請自行標示小題題號

1.1

```
module dataflow #(
    parameter n=16
)(
    input  [n-1:0] a,b,
    input          ci,
    output [n-1:0] s,
    output         co
);
    assign {co,s} = {1'b0,a} + {1'b0,b} + ci;
endmodule
```

1.2

```
module structural #C
    parameter n=16
)C
    input  [n-1:0] a,b,
    input          ci,
    output [n-1:0] s,
    output         co
);
    wire [n:0] c;
    assign c[0] = ci;
    assign co = c[n];
    genvar i;
    generate
        for(i=0; i<n; i=i+1) begin :for_loop
            FA fa(c[i+1], s[i], a[i], b[i], c[i]);
        end
    endgenerate
endmodule
```

1.3

```
module behavioral #C
    parameter n=16
)C
    input  [n-1:0] a,b,
    input          ci,
    output reg [n-1:0] s,
    output reg         co
);
    always @(*) begin
        {co,s} = {1'b0,a} + {1'b0,b} + ci;
    end
endmodule
```

*要用 reg

A-2 Different Modeling Methods — 2-to-1 Multiplexer

Original weight: 15%

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

Use different modeling methods (dataflow, structural, behavioral) to describe a two-to-one multiplexer with 1-bit inputs named `in0`, `in1`, 1-bit select signal `s`, and 1-bit output named `out` so that the output is `in1` when `s=1` and the output is `in0` when `s=0`. Write a complete Verilog code.

2.1 Dataflow using conditional operator.

2.2 Structural using Verilog built-in gate primitive `bufif1`. Hint: `bufif1 instance_name(out, in, control);` `out=in` if `control=1`, otherwise `out=1'bz`.

2.3 Behavioral using `if ... else ...`.

ANSWER / 作答區 請自行標示小題題號

2.1
```
module dataflow (
    input in0, in1, s,
    output out
);
    assign out = s ? in1 : in0;
endmodule
```

2.2
```
module structural ( input in0, in1, s, output out );
    wire ns;
    not (ns, s)
    bufif1 (out, in0, ns);
    bufif1 (out, in1, s);
endmodule
```

2.3
```
module behavioral (input in0, in1, s, output reg out );
    always @(*) begin
        if(s) begin
            out = in1;
        end
        else begin
            out = in0;
        end
    end
endmodule
```

behavioral 用 reg

A-3 Different Modeling Methods — 4-to-1 Multiplexer

Original weight: 15%

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

Use different modeling methods (dataflow, structural, behavioral) to describe a four-to-one multiplexer with 16-bit inputs named `in0`, `in1`, `in2`, `in3`, 2-bit select signal `s`, and 16-bit output named `out`. Write a complete Verilog code.

3.1 Dataflow using a “nested” conditional operator.

3.2 Structural assuming a two-to-one multiplexer `MUX2` is available.

3.3 Behavioral using multi-way branch `case ... endcase`.

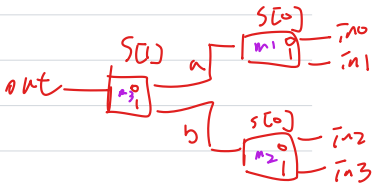
s	in
00	0
01	1
10	2
11	3

練習介面約定（沿用題庫）：3.2 的 MUX2 資料埠為 16-bit，埠名為 out、in0、in1、s。

ANSWER / 作答區 請自行標示小題題號

3.1
```
module dataflow (input [15:0] in0, in1, in2, in3, input [1:0] s, output [15:0] out);
    assign out = s[1] ? (s[0] ? in3 : in2) :
                        (s[0] ? in1 : in0);
endmodule
```

3.2
```
module structural (input [15:0] in0, in1, in2, in3, input [1:0] S, output [15:0] out);
    wire [15:0] a, b;
    MUX2 m1 (a, in0, in1, s[0]);
    MUX2 m2 (b, in2, in3, s[0]);
    MUX2 m3 (out, a, b, s[1]);
endmodule
```



3.3
```
module behavioral (input [15:0] in0, in1, in2, in3, input [1:0] S, output reg [15:0] out);
    always @ (*) begin
        case (S)
            2'b00 : out = in0;
            2'b01 : out = in1;
            2'b10 : out = in2;
            2'b11 : out = in3;
            default : out = 16'b0;
        endcase
    end
endmodule
```

A-4 Flip-Flop (FF)

Original weight: 20%

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

4.1 Use Verilog to describe a flip-flop with clock signal `clk`, data input `d`, output `q`.

4.2 Write a Verilog code segment to describe an active-high asynchronous resettable flip-flop with clock signal `clk`, data input `d`, reset signal `rst`, output `q_async`.

4.3 Write a Verilog code segment to describe an active-high synchronous resettable flip-flop with clock signal `clk`, data input `d`, reset signal `rst`, output `q_sync`.

4.4 Draw the waveforms of `clk`, `d`, `rst`, `q_async`, `q_sync` to clearly show the differences of async and sync resettable FF. (Hint: draw in one plot.)

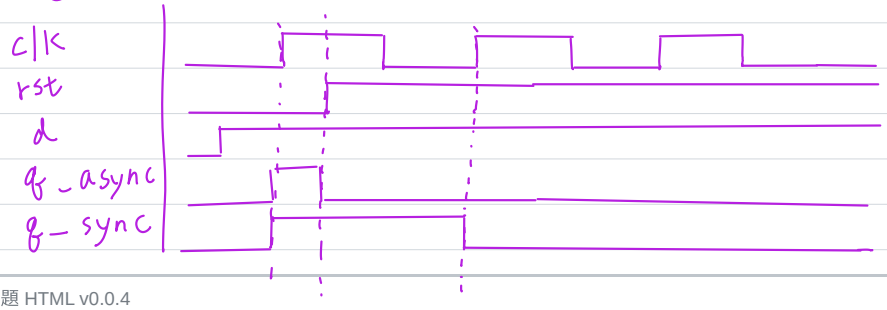
ANSWER / 作答區 請自行標示小題題號

4.1
```
module DFF(input d, clk, output reg q);
    always @(posedge clk) begin
        q <= d;
    end
endmodule
```

4.2
```
module DFF(input clk, d, rst, output reg q_async);
    always @(posedge clk, posedge rst) begin
        if(rst) begin
            q_async <= 1'b0;
        end
        else begin
            q_async <= d;
        end
    end
endmodule
```

4.3
```
module DFF(input clk, d, rst, output reg q_sync);
    always @(posedge clk) begin
        if(rst) begin
            q_sync <= 1'b0;
        end
        else begin
            q_sync <= d;
        end
    end
endmodule
```

4.4



A-5 Clock-Gating

Original weight: 20%

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

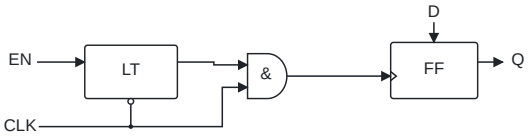
5.1 Draw the expected synthesis results of the following code segment.

```
assign GCLK = CLK & EN;
always @(posedge GCLK)
    Q <= D;
```

5.2 Draw the expected synthesis results of the following code segment.

```
always @(posedge CLK)
    if (EN) Q <= D;
```

5.3 The above two clock-gating methods will be synthesized to the shown robust and efficient circuit by providing proper Synopsys synthesis scripts. Write a Verilog code segment to describe the robust clock-gating registers.



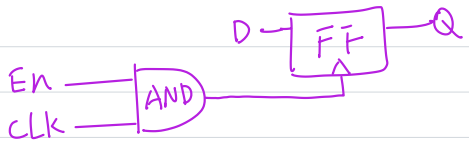
5.4 Write a complete Verilog code to model `d=(a+b)*c` with two pipelined stages where inputs a, b, c are 8-bit and output d is 16-bit. The first stage performs addition and the second stage performs multiplication. Use the clock-gate code shown in 5.1 to turn off multiplication in the 2nd stage in case `c=0`.

5.4 作答約定：依老師複習 PDF 第 35 頁的版本作答。

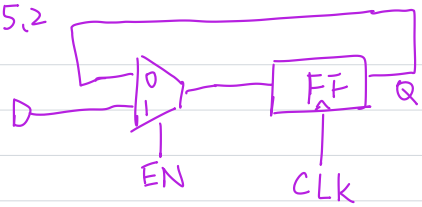
ANSWER / 作答區

請自行標示小題題號

5.1



5.2



5.3

```
module clock_gating (input EN, CLK, D, output reg Q);
  reg en_latched;
  wire gclk;

  always @(*) begin
      if (~CLK) begin
          en_latched = EN;
      end
  end
  assign gclk = CLK & en_latched;

  always @(posedge gclk) begin
      Q <= D;
  end
endmodule
```

5.4 module clock_gating (input [7:0] a,b,c, input clk, output [15:0] d);

reg [8:0] sum_reg ★是8
reg [7:0] c_reg;
reg [15:0] product_reg; ★3個 reg
reg en_latched, zero_out;
wire gclk; 2個 flag_reg, 1個 wire

always @(posedge clk) begin
sum_reg <= {1'b0,a} + {1'b0,b};
c_reg <= c;
zero_out <= (c_reg==0);
end

★第二階用到的 zero_out 和 en_latched 應該用 c_reg

always @(*) begin
if(~clk) en_latched = (c_reg!=0);
end

assign gclk = clk & en_latched;
always @(posedge gclk) begin
product_reg <= c_reg * sum_reg;
end

是否停用 stage 2?

assign d = zero_out ? 16'b0 : product_reg;
endmodule

↑停 ↑沒停

A-6 Non-blocking / Blocking / Generic Shift Registers

Original weight: 15%

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

6.1 What is the expected synthesis result for the following code segment with non-blocking assignments?

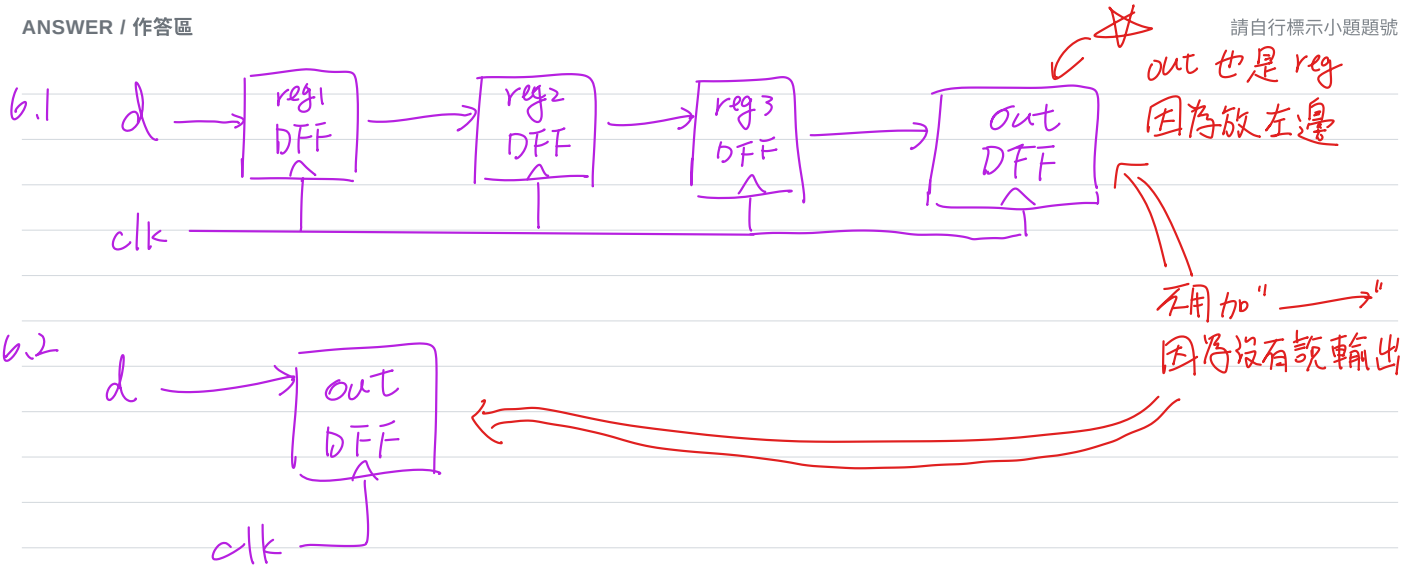
```
always @(posedge clk) begin
    reg1 <= d;
    reg2 <= reg1;
    reg3 <= reg2;
    out  <= reg3;
end
```

6.2 What is the expected synthesis result for the following code segment with blocking assignments?

```
always @(posedge clk) begin
    reg1 = d;
    reg2 = reg1;
    reg3 = reg2;
    out  = reg3;
end
```

6.3 Write a code segment to describe generic shift registers with `n=8` registers where each register has `m=16`.

ANSWER / 作答區 請自行標示小題題號



6.3

```
module shift_registers #(parameter n=8,m=16)(
    input clk,
    input [m-1:0] d,
    output [m-1:0] out
);
    reg [m-1:0] r [0:n-1];
    integer i;
    always @(posedge clk) begin
        r[0]<=d;
        for(i=1;i<n;i=i+1) begin
            r[i]<=r[i-1];
        end
    end
    assign out=r[n-1];
endmodule
```

i, i-1

B-1 Unsigned and Signed

Original weight: 10%

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

Let `a=4'b1010` and `b=8'b0010_0000`.

1.1 (5%) What is the binary representation of the 8-bit number `c` where `c=a+b`, if `a`, `b`, `c` are all declared as wire type?

1.2 (5%) What is the binary representation of the 8-bit number `c` where `c=a+b`, if `a`, `b`, `c` are all declared as signed type?

ANSWER / 作答區 請自行標示小題題號

1.1 32, 16, 8, 4, 2, 1

a = 10, b = 32, C = 10 + 32 = 42 ⇒ C = 8'b0010_1010 #

1.2

a = -8 + 2 = -6, b = 32, C = 32 - 6 = 26 ⇒ C = 8'b0001_1010 #

B-2 Logical Shift and Arithmetic Shift

Original weight: 10%

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

題面註記（沿用題庫）：原題 16'd-32 按 b = −32 解讀；程式中採 -16'sd32。

Let `a, b, c, d` be declared as 16-bit signed type and `a=16'd32, b=16'd-32`.

2.1 (5%) What is the binary representation and the corresponding decimal value of `c = a >> 3`, and what is the decimal value of `d = b >> 3`?

2.2 (5%) What is the binary representation and the corresponding decimal value of `c = a >>> 3`, and what is the decimal value of `d = b >>> 3`?

ANSWER / 作答區 請自行標示小題題號

2.1

64, 32, 16, 8, 4, 2, 1

a = 32(10) = ...0100000(2)

b = -32(10) = ...1100000(2)

c = a>>3 = 0000_0000_0000_0100(2) = 4(10) #

d = b>>3 = 0001_1111_1111_1100(2) = $2^{14}-1-3 = 8172-4 = 8188_{(10)}$ #

2.2

c = a>>>3 = 0000_0000_0000_0100(2) = 4(10)

d = b>>>3 = 1111_1111_1111_1100(2) = -4(10)

B-3 Pipelined Multiply-Add (MAD)

Original weight: 10%

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

Write a complete Verilog code to describe the pipelined MAD `d=a*b+c` with 8-bit inputs `a`, `b`, and 16-bit input `c`, where the first stage performs multiplication and the second stage performs addition. Use active-low asynchronous-reset registers for the pipeline registers. Your code should look like the following:

```
module MAD(
    input [7:0] a, b,
    input [15:0] c,
    input clk, reset_n,
    output reg [15:0] d
);
    ...
endmodule
```

ANSWER / 作答區 請自行標示小題題號

```
B-3 module MAD(
        input [7:0] a, b,
        input [15:0] c,
        input clk, reset_n,
        output reg [15:0] d
    );
        reg [15:0] product_reg;
        reg [15:0] c_reg;

        always @(posedge clk, negedge reset_n) begin
            if(!reset_n) begin
                product_reg <= 16'b0;
                c_reg <= 16'b0;
                d <= 16'b0;
            end
            else begin
                product_reg <= a*b;
                c_reg <= c;
                d <= product_reg + c_reg;
            end
        end
    end module
```

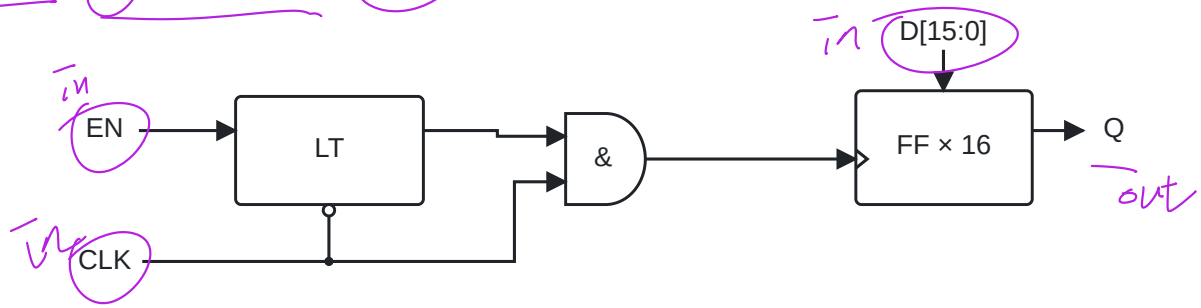
B-4 Clock-Gating / Latch + FF Circuit

Original weight: 10%

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

Write Verilog code segment to describe the shown circuit where `LT` is a latch; `FF` is flip-flop; register `Q` contains 16 flip-flops (FFs); `D` is a 16-bit data signal; `CLK` is the clock signal. What is the function of the circuit?



ANSWER / 作答區 請自行標示小題題號

```
module A (input EN, CLK, input [15:0] D, output reg [15:0] Q);
    reg en_latched;
    wire gclk;

    always @(*) begin
        if (!CLK) en_latched = EN;
    end

    assign gclk = CLK & en_latched;

    always @(posedge gclk) begin
        Q <= D;
    end
endmodule
```

The circuit can prevent the glitch to affect the Flip-Flop.

B-5 Ripple Carry Adder (RCA)

pure：纯 procedural statements：程序式(非电路式) → behavioral → always (*)

Original weight: 10%

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

5.1 (5%) Use pure procedural statements inside an always block to model a 32-bit ripple carry adder. Write a complete Verilog module as follows.

```
module RCA32_beh(
    output reg Cout,
    output reg [31:0] S,
    input Cin,
    input [31:0] A, B
);
    ...
endmodule
```

5.2 (5%) Describe a 32-bit RCA using structural modeling method with module instantiation of the given pre-defined module FA. Hint: use generate loop for repeated module instantiation.

```
module FA(output co, s, input a, b, c);
    assign {co, s} = a + b + c;
endmodule

module RCA32_str(
    output Cout,
    output [31:0] S,
    input Cin,
    input [31:0] A, B
);
    ...
endmodule
```

ANSWER / 作答區 請自行標示小題題號

5.1

```
module RCA32_beh (
    output reg Cout,
    output reg [31:0] S,
    input Cin,
    input [31:0] A,B
);
    reg [32:0] C;

    integer i;
    always @(*) begin
        C[0] = Cin;
        for(i=0; i<32; i=i+1) begin
            C[i+1] = (A[i]& B[i])|(B[i] & C[i])|(C[i]&A[i]);
            S[i] = A[i]^ B[i]^ C[i];
        end
        Cout = C[32];
    end
endmodule
```

※是A不是a，然後要有[i]（原本寫 a、b）

→放最後面，因為不是並行的

5.2

```
module RCA32_str(
        output      Cout,
        output      [31:0]S,
        Input  Cin,
        Input  [31:0]A,B
);
        wire  [32:0]  C;
        assign  c[0]=cin;
        assign  Cout = C[32];
        genvar i;

        generate
            for(i=0; i<32; i=i+1) begin: Loop
                FA  fa(C[i+1], S[i], A[i], B[i], C[i]);
            end
        endgenerate
endmodule
```

B-6 Shift Registers

Original weight: 15%

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

6.1 (5%) Write Verilog code-segment to describe shift registers with 4 registers `reg0, reg1, reg2, reg3`, where each register has 16 bits. Let `d` be the input and `reg3` be the output.

6.2 (10%) Write Verilog code segment to describe shift registers with `n` registers `x[0:n-1]` where each register has `bw` bits. Use parameter for `n` and `bw`. Let `d` be the input and `x[n-1]` be the output of the shift registers.

ANSWER / 作答區 請自行標示小題題號

6.1
```
module A( input [15:0] d, input clk, output reg [15:0] reg3);
    reg [15:0] reg0, reg1, reg2;
    always @(posedge clk) begin
        reg0 <= d;
        reg1 <= reg0;
        reg2 <= reg1;
        reg3 <= reg2;
    end
endmodule
```
在左邊的就是 DFF，有4個，共4階。

6.2
```
module A #(parameter n=8, bw=16) (
    input [bw-1:0] d, input clk, output [bw-1:0] out
);
    reg [bw-1:0] x [0:n-1];
    integer i;

    always @(posedge clk) begin
        x[0] <= d;
        for(i=0; i<n-1; i=i+1) begin
            x[i+1] <= x[i];
        end
    end
    assign out = x[n-1];
endmodule
```
parameter 一定要有初始值

放前面

B-7 Dot Product (DP)

Original weight: 15%

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

題面註記（沿用題庫）：7.2 的 Module Header 名稱位置有誤；作答時將 DPn 放在 #(...) 前。

7.1 (5%) Write a complete Verilog code to describe dot-product of two vectors `[a0, a1, a2, a3]` and `[b0, b1, b2, b3]` where `ai` and `bi` are all 16-bit numbers. Let the output `y=a0*b0+a1*b1+a2*b2+a3*b3` be a 32-bit number.

```
module DP4(
    output reg [31:0] y,
    input [15:0] a0, a1, a2, a3,
    b0, b1, b2, b3
);
    ...
endmodule
```

7.2 (10%) Write a complete Verilog code for a generic dot-product of two vectors where each vector contains n elements with each element of bw-bit. Use parameter for n and bw. Note that module input/output ports cannot be array type, so use long bit-vectors as module input ports.

```
module #(parameter n=8, bw=16) DPn(
    output reg [2*bw-1:0] y,
    input [bw*n-1:0] long_a, long_b
);
    ...
endmodule
```

ANSWER / 作答區 請自行標示小題題號

7.1

```
module DP4(
    output reg [31:0] y,
    input [15:0] a0, a1, a2, a3,
    b0, b1, b2, b3
);

    always @(*) begin

        y = a0*b0 + a1*b1 + a2*b2 + a3*b3;

    end
endmodule
```

7.2

```
module DPn #(parameter n=8, bw=16)(
    output reg [2*bw-1:0] y,
    input [bw*n-1:0] long_a, long_b
);
    integer i;

    always @(*) begin
        y = 2*bw'b0;
        for(i=0; i<n; i=i+1) begin
            y = y + (long_a[bw*i +: bw] * long_b[bw*i +: bw]);
        end
    end
endmodule
```

不能是 parameter → 改成 `y = {2*bw{1'b0}};`

(位元)'(base)(value)

B-8 Verilog Code Tracing

Original weight: 20%

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

題面註記（沿用題庫）：8.1 的 Function 宣告含型別筆誤；呼叫的數字按 8-bit 二進位位元樣式解讀。以下保留原題運算內容。

8.1 (10%) Describe the operation of the following Verilog code function `fx`. What is the returned value of function call `fx(1000_0001, 1000_0010)`? What is the returned value of `fx(0111_1100, 0000_0100)`? What is the returned value of `fx(0001_0101, 0010_1010)`?

```
function signed [7:0] fx(
    input wire signed [7:0] a, b
);
    wire signed [7:0] result;
    begin
        result = a + b;
        if ((a[7] == b[7]) &
            (result[7] != a[7]))
            fx = {a[7], {7{~a[7]}}};
        else
            fx = result;
    end
endfunction
```

8.2 (10%) What is the value of y if `din = 16'b0001_0100_1101_1111`? What is the value of y if `din = 16'b1111_1001_1101_1111`? What is the value of y if `din = 16'b0000_0000_0000_0000`? What is the function of the Verilog code?

```
`define TRUE  1'b1
`define FALSE 1'b0

module problem(
    input [15:0] din,
    output reg [4:0] y
);
    reg continue;
    integer i;
    always @(*) begin
        continue = `TRUE;
        for (i=15; continue && (i>=1);
             i=i-1) begin
            y = din[i] != din[i-1]
              ? 5'd16-i : 5'd16;
            continue = din[i] != din[i-1]
                     ? `FALSE : `TRUE;
        end
    end
endmodule
```

15~1
相, y=

ANSWER / 作答區 請自行標示小題題號

8.1 ① if overflow, fx will return the maximum number (01111111) or the minimum number (10000000), if no overflow, fx will return a+b.

② 1000_0001 + 1000_0010 = 0000_0011 → overflow → returned value = 1000_0000 #

③ 0111_1100 + 0000_0100 = 1000_0000 → overflow → returned value = 0111_1111 #

④ 0001_0101 + 0010_1010 = 0011_1111 → no overflow → returned value = 0011_1111 #

① din = 0001_0100_1101_1111

0 1 2 3

⇒ y = 3 = 0000_0000_0000_0011$_{(2)}$ #

② din = 1111_1001_1101_1111

0 1 2 3 4 5

⇒ y = 5 = 0000_0000_0000_0101$_{(2)}$ #

③ din = 0000_0000_0000_0000

16

⇒ y = 16 = 0000_0000_0001_0000$_{(2)}$ #

④ Frome left to right find the first two different bits, and y will be the index (begin from 0) of the right one.

If there has no different between every bits, y will be 16$_{(10)}$.

C-1 Enabled 2-to-4 Decoder

Verilog-2001/2005｜C-10 以外的功能測試假設為已知 0 / 1 輸入

Name: _____ Date: _____

C-1.1 Dataflow

Design a combinational 2-to-4 decoder with a 2-bit input `a`, a 1-bit enable `en`, and a 4-bit output `y`. When `en=0`, all outputs are zero. When `en=1`, only `y[a]` is one. Assume binary inputs.

Write a complete module named `decoder_df` using four Boolean continuous assignments.

C-1.2 Structural

Implement the same decoder as a complete module named `decoder_str`. Use only built-in `not` and `and` primitives.

C-1.3 Behavioral

Implement the decoder as a complete module named `decoder_bh` using `always @(*)` and `case`. Assign all output bits for every input condition.

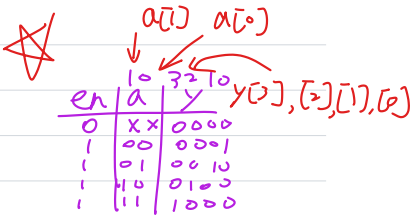
ANSWER / 作答區 請自行標示小題題號

1-1

```
module decoder_df ( input [1:0] a , input en , output [3:0] y );

    assign y[3] = en & a[1] & a[0] ;
    assign y[2] = en & a[1] & (~a[0]);
    assign y[1] = en & (~a[1]) & a[0] ;
    assign y[0] = en & (~a[1]) & (~a[0]) ;

endmodule
```



1-2

```
module decoder_str ( input [1:0] a, input en, output [3:0] y );

    wire en_n, a0_n, a1_n ;
    not (en_n, en);
    not (a0_n, a[0]);
    not (a1_n, a[1]);
    and (y[3], en, a[1], a[0]);
    and (y[2], en, a[1], a0_n );
    and (y[1], en, a1_n, a[0]);
    and (y[0], en, a1_n, a0_n);
endmodule
```

1-3

```
module decoder_bh (input [1:0] a, input en, output reg [3:0] y);
    always @ (*) begin
        case ({a,en})
            3'b001: y = 4'b0001 ;
            3'b011: y = 4'b0010 ;
            3'b101: y = 4'b0100 ;
            3'b111: y = 4'b1000 ;
            default: y = 4'b0000 ;
        endcase
    end
endmodule
```

C-2 DEMUX: Zero versus High Impedance

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

C-2.1 Zero-filled DEMUX

Write a complete behavioral module `demux4_zero` with one-bit input `d`, two-bit input `sel`, and four-bit output `y`. Route `d` to `y[sel]` and drive every unselected output to zero. Assume binary inputs.

C-2.2 Tri-state DEMUX

Write a complete module `demux4_z` for the same routing function, but make every unselected output high impedance. Use four `bufif1` primitives; continuous assignments may generate their controls.

C-2.3 Output values

For each input below, give `y[3:0]` for both implementations. Are the two implementations equivalent on the unselected outputs?

d	sel
1	01
0	10
1	11

ANSWER / 作答區 請自行標示小題題號

2.1
```
module demux4_zero (input d, input [1:0] sel, output reg [3:0] y);
    always @(*) begin
        y = 4'b0000;
        y[sel] = d;
    end
endmodule
```

2.2
```
module demux4_2 (input d, input [1:0] sel, output [3:0] y);
    wire [3:0] ctr;
    assign ctr = {sel==2'd3, sel==2'd2, sel==2'd1, sel==2'd0};
    bufif(y[3], d, ctr[3]);
    bufif(y[2], d, ctr[2]);
    bufif(y[1], d, ctr[1]);
    bufif(y[0], d, ctr[0]);
endmodule
```

2.3

d	sel	demux4_zero	demux4_z
1	01	y=0010	y=zz1z
0	10	y=0000	y=z0zz
1	11	y=1000	y=1zzz

The two implementations are not equivalent on the unselected outputs.

C-3 Four-bit Rotate in Three Styles

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

C-3.1 Dataflow

Design a combinational four-bit rotator. With `left=1`, rotate `data[3:0]` left by one bit; with `left=0`, rotate it right by one bit. The bit shifted out must wrap around. Assume binary inputs. Do not use a clock.

Write a complete module `rotate4_df` using a conditional operator and concatenations.

C-3.2 Structural

Write `rotate4_str` using four instances of the provided one-bit module `mux2_g(y, d0, d1, s)`. It selects `d0` when `s=0` and `d1` when `s=1`. Do not redefine `mux2_g`.

C-3.3 Behavioral

Write a complete module `rotate4_bh` using `always @(*)` and `if ... else`.

C-3.4 Trace the rotation

For `data=4'b1001`, give the output for `left=1` and `left=0`.

ANSWER / 作答區 請自行標示小題題號

3.1
```
module rotate4_df (input left, input [3:0] data, output [3:0] out);
    assign out = (left) ? {data[2:0], data[3]}
                        : {data[0], data[3:1]};
endmodule
```

3.2
```
module rotate4_str (input left, input [3:0] data, output [3:0] out);
    mux2_g m1(out[3], data[0], data[2], left);
    mux2_g m2(out[2], data[3], data[1], left);
    mux2_g m3(out[1], data[2], data[0], left);
    mux2_g m4(out[0], data[1], data[3], left);
endmodule
```

3.3
```
module rotate4_bh (input left, input [3:0] data, output reg [3:0] out);
    always @(*) begin
        if (left) out = {data[2:0], data[3]};
        else      out = {data[0], data[3:1]};
    end
endmodule
```

3.4
```
left=1 ⇒ out=4'b0011 #
left=0 ⇒ out=4'b1100 #
```

C-4 Highest-bit Priority Encoder

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

C-4.1 Behavioral encoder

Write a complete module `priority4`. The four-bit input `in` may contain multiple ones. Output the position of the highest set bit as two-bit `code`. Set `valid=1` if any bit is one. For `in=0000`, output `valid=0` and `code=0`. Assume binary inputs.

C-4.2 Boolean equations

Give dataflow assignments for the same function. For this separate implementation, assume `valid` and `code` are wires.

C-4.3 Multiple requests and no request

Give `valid` and `code` for each input: `1010`, `0101`, `0011`, `0001`, and `0000`.

ANSWER / 作答區 請自行標示小題題號

4.1

```
module priority4 (input [3:0] in, output reg [1:0] code, output reg valid);

    always @ (*) begin
        valid = |in;
        if      (in[3]) code = 2'd3;
        else if (in[2]) code = 2'd2;
        else if (in[1]) code = 2'd1;
        else            code = 2'd0;
    end
endmodule
```

4.2

```
module priority4 (input [3:0] in, output [1:0] code, output valid);

    assign valid = |in;
    assign code[1] = in[3] | in[2];
    assign code[0] = in[3] | (~in[2] & in[1]);

endmodule
```

in	valid	code
1010	1	11
0101	1	10
0011	1	01
0001	1	00
0000	0	00

C-5 Eight-bit Even-parity Generator

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

C-5.1 Reduction operator

Design a parity generator for eight-bit binary input `data`. Choose one-bit `p` so that `{data,p}` contains an even number of ones. Write a complete dataflow module `parity8_df` using a reduction operator.

C-5.2 XOR-tree structure

Write a complete structural module `parity8_str` using only two-input `xor` primitives. Draw a balanced XOR tree and state the number of gates and logic levels.

C-5.3 Procedural loop

Write a complete behavioral module `parity8_bh` with a procedural `for` loop. Do not use a clock or reduction XOR in this version.

C-5.4 Parity values

Give p for `data=00000111`, `00111100`, and `11111111`.

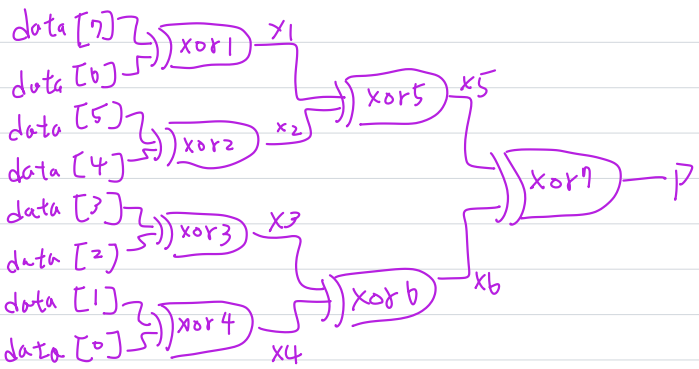
ANSWER / 作答區 請自行標示小題題號

5.1

```
module parity8_df ( input [7:0] data , output p );

    assign p=^data;
endmodule
```

5.2



Number of gates = 7

logic levels = 3

```
module parity8_str ( input [7:0] data , output p );

    wire x1,x2,x3,x4,x5,x6;
    xor (x1, data[7], data[6]);
    xor (x2, data[5], data[4]);
    xor (x3, data[3], data[2]);
    xor (x4, data[1], data[0]);
    xor (x5, x1, x2);
    xor (x6, x3, x4);
    xor (p, x5, x6);
endmodule
```

5.3

```
module parity8_bh ( input [7:0] data , output reg p );

    integer i;
    always @ (*) begin
        p = 1'b0 ;
        for(i=0; i<8; i=i+1)begin
            p = p^ data[i];
        end
    end
endmodule
```

※ 要先初始化

C-6 Four-bit ALU and Zero Flag

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

C-6.1 Complete ALU

Write a complete behavioral module `alu4` with four-bit unsigned inputs `a` and `b`, two-bit input `op`, four-bit output `y`, and one-bit output `zero`. Select AND, OR, addition, and subtraction for `op=00,01,10,11`, respectively. Keep only the low four result bits. Set `zero=1` exactly when `y=0`. Use `y=0` for a case mismatch. Do not add carry or overflow ports.

C-6.2 Trace and truncation

Give `y[3:0]` and `zero` for each row.

a	b	op
1100	1010	00
1100	1010	01
1111	0001	10
0010	0101	11
0011	0011	11

ANSWER / 作答區 請自行標示小題題號

6.1

```
module alu4 (
    input [3:0] a, b, input [1:0] op, output reg [3:0] y, output zero
);
    always @(*) begin
        case (op)
            2'b00: y = a & b;
            2'b01: y = a | b;
            2'b10: y = a + b;
            2'b11: y = a - b;
            default: y = 0;
        endcase
    end
    assign zero = ~|y;
endmodule
```

6.2

a	b	op	y[3:0]	zero
1100	1010	00	1000	0
1100	1010	01	1110	0
1111	0001	10	0000	1
0010	0101	11	1101	0
0011	0011	11	0000	1

減法定 $A-B = A+(\sim B+1)$

如何解讀結果就看是否為 signed

C-7 Leading-zero Counter

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

C-7.1 Complete LZC

Write a complete combinational module `lzc4` that counts consecutive zeros from the MSB of four-bit binary input `data`. Output `count=4` for `0000`. Choose the minimum output width that represents all results and justify it.

C-7.2 Boundary cases

Give `count` for `1000`, `0101`, `0010`, `0001`, `0000`, and `1111`.

ANSWER / 作答區 請自行標示小題題號

7.1

```
module lzc4 ( input [3:0] data, output reg [2:0] count );

always @ (*) begin

    if(data[3]) count = 3'd0;
    else if (data[2]) count = 3'd1;
    else if (data[1]) count = 3'd2;
    else if (data[0]) count = 3'd3;
    else count = 3'd4;

end
endmodule
```

results require from 0 to 4, so need 3bit can represents from 0 to 7.

7.2

data	y
1000	0
0101	1
0010	2
0001	3
0000	4
1111	0

C-8 Three-state Moore FSM with Enable

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

C-8.1 State-transition diagram

Design a three-state FSM. Use `S0=2'd0`, `S1=2'd1`, and `S2=2'd2`. An active-low asynchronous reset `rst_n` resets the state to S0. At each rising edge of `clk`, advance S0→S1→S2→S0 if `en=1`, otherwise hold the state. Output `y=1` only in S2. Recover the unused encoding `2'b11` to S0 at the next rising edge, regardless of `en`. Assume binary inputs.

Draw the state-transition diagram. Show the output in each state, enable conditions, reset, and unused-state recovery.

C-8.2 Behavioral implementation

Write a complete module `fsm3(input clk, rst_n, en, output reg y)`. Separate the clocked state register from combinational next-state and output logic.

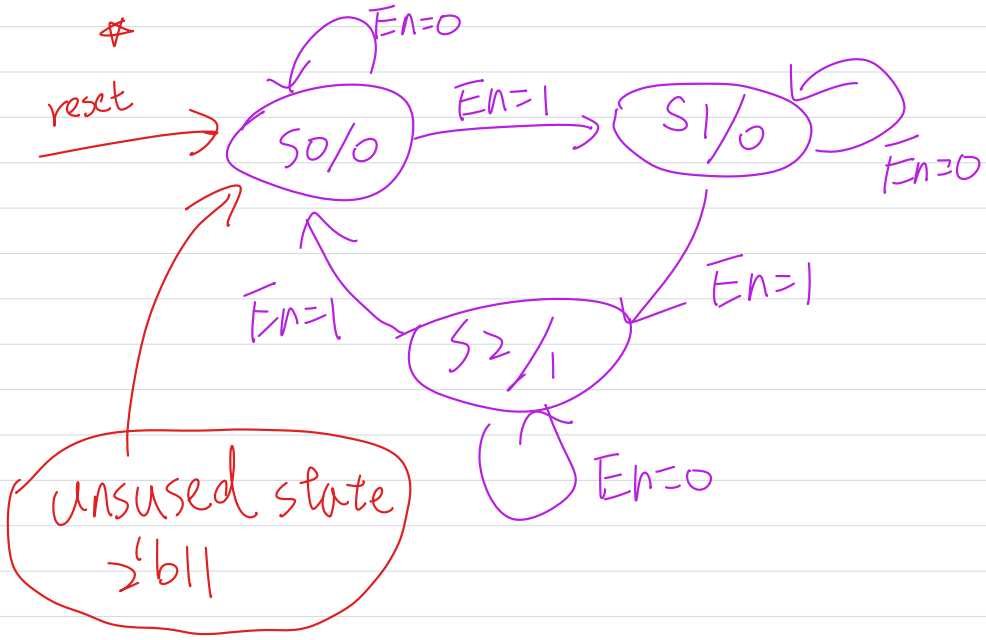
C-8.3 Trace eight edges

After reset, apply `en=1,0,1,0,0,1,1,1` at eight successive rising edges. List the state and `y` after each edge, after nonblocking updates and combinational settling.

C-8.4 Moore versus a pulse

Is this a Moore or Mealy FSM? Is `y` guaranteed to be high for only one clock cycle? Explain briefly.

ANSWER / 作答區 請自行標示小題題號



```
module fsm3 (
    input clk, rst_n, en,
    output reg y
);
    parameter s0 = 2'd0, s1 = 2'd1, s2 = 2'd2;
    reg [1:0] state, next_state;

    always @ (posedge clk, negedge rst_n) begin
        if(~rst_n) state <= s0;
        else state <= next_state;
    end

    always @ (*) begin
        case(state)
            s0: begin
                if(en) next_state = s1;
                else next_state = s0;
            end
            s1: begin
                if (en) next_state = s2;
                else next_state = s1;
            end
            s2: begin
                if(en) next_state = s0;
                else next_state = s2;
            end
            default : next_state = s0;
        endcase
        y = (state == s2);
    end
endmodule
```

其實不用 else 因為會記住上次的

Recover the unused encoding 2'b11 to s0

8.3

en	state	y
1	S1	0
0	S1	0
1	S2	1
0	S2	1
0	S2	1
1	S0	0
1	S1	0
1	S2	1

8.4

① moore FSM

② NO, when en be 0, the state will be not change, y will be not change.

C-9 Hardware Inference and Combinational Repairs

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

C-9.1 Infer the hardware

Assume all signals below are one bit. Identify and draw the hardware inferred by each fragment. Does the keyword `reg` alone imply a flip-flop?

```
// Fragment A
reg y;
always @(*) y = a & b;

// Fragment B
reg q;
always @(*)
    if (en) q = d;
```

C-9.2 Missing sensitivity

Consider `always @(a) y = a & b;`. The block has just evaluated with `a=1` and `b=0`, so `y=0`. Now only b changes to 1. What does this RTL simulation leave in y? What should a combinational AND output be? Rewrite the block correctly.

C-9.3 Remove unintended storage

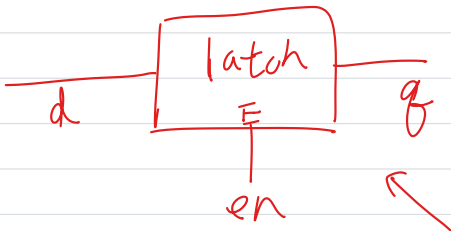
The required function is now `q=d` when `en=1`, and `q=0` when `en=0`. Rewrite Fragment B as combinational logic. Is changing the sensitivity list alone sufficient?

ANSWER / 作答區 請自行標示小題題號

9.1 Fragment A



Fragment B



latch 本身可有 enable
(F-F 也可有)

Fragment A and B both not imply a flip-flop

9.2

① y = 0

② y = 1

③ always @(*) y = a & b;

9.3

① reg q;

```
always @(*) begin
    if(en) q <= d;
    else   q <= 1'b0;
end
```

② Not enough, should add else block under "if (en)".

C-10 Operators, Four-state Equality, and Case Matching

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

C-10.1 Result values and widths

Let `a=4'b1010` and `b=4'b0101` be unsigned four-bit vectors. For each expression, give the result width before any assignment extension and its bit pattern: `a & b`, `a && b`, `~a`, `!a`, `^a`, `|a`, `{a[1:0],b[1:0]}`, `{4{b[0]}}`, and `2'd11`.

C-10.2 Equality with unknown bits

In four-state simulation, evaluate the following expressions. Give a one-bit result of 0, 1, or X.

Expression
`4'b10x1 == 4'b1001`
`4'b10x1 === 4'b1001`
`4'b10x1 !== 4'b1001`
`4'b10x1 == 4'b10x1`
`4'b10x1 === 4'b10x1`
`4'b1x01 == 4'b0x01`

C-10.3 case / casez / casex

Assume two-bit `sel` and `reg [1:0] tag` inside a combinational block. Apply the items below independently using `case`, `casez`, and `casex`. For `sel=2'b1x`, `2'b1z`, and `2'b11`, give the selected value of tag. Keep the item order shown. Why can casex hide a real unknown?

```
// Replace only the opening keyword for the
other two versions.
case (sel)
    2'b10: tag = 2'd1;
    2'b11: tag = 2'd2;
    default: tag = 2'd3;
endcase
```

ANSWER / 作答區

請自行標示小題題號

10-1 a&b ⇒ 0000, 4bits

a&&b ⇒ 1, 1bit

~a ⇒ 0101, 4bits

!a ⇒ 0, 1bit

^a ⇒ 0, 1bit

|a ⇒ 1, 1bit

{a[1:0],b[1:0]} ⇒ 1001

{4{b[0]}} ⇒ 1111

2'd11 ⇒ 11, 2bit

10-2

① X

② 0

③ 1

④ X

⑤ 1

⑥ 0

1x01
0x01
↳不同了不用看到 x 所以是0 ✱

10-3

sel	case	casez	casex
1x	3	3	1
1z	3	1	1
11	2	2	2

z和? → 當萬用字元

z和?和x → 當萬用字元

✱不一定是case中放x,z它是比對,所以sel中有x,z也有用

因為casex會把x當萬用字元

C-11 Signedness after Selection and Truncation

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

C-11.1 Same bits, different expressions

For the declarations and continuous assignments below, give the final bit pattern and signed decimal value of each named result. Explain why from_slice and from_concat differ from whole even though they start with the same five bits.

```
wire signed [4:0] x;
wire signed [4:0] whole, from_slice, from_concat;
wire signed [3:0] narrow, trunc_then_shift, shift_then_trunc;

assign x               = 5'b10101;
assign whole           = x >>> 1;
assign from_slice      = x[4:0] >>> 1;
assign from_concat     = {x} >>> 1;
assign narrow          = x[3:0];
assign trunc_then_shift= narrow >>> 1;
assign shift_then_trunc = x >>> 1;
```

C-11.2 Signed comparison

Let `u=2'b11` and `v=2'b01` both be declared `wire [1:0]`. Let `s=2'b11` and `t=2'b01` both be declared `wire signed [1:0]`. Evaluate `u > v` and `s > t`, and state the operand values.

C-11.3 Carry is not signed overflow

For each four-bit input pair below, compute the low four sum bits, the carry-out from a zero-extended five-bit addition, and signed overflow when the original operands are interpreted as four-bit two's-complement values. Do not saturate the result. Use `(a,b)=(0111,0001)` and `(1111,0001)`. Are carry-out and signed overflow interchangeable?

ANSWER / 作答區 請自行標示小題題號

11.1 ①

x = 5'b10101 = -11(10), 10101(2)

whole = 5'b11010 = -6(10), 11010(2)

from_slice = 5'b01010 = 10(10), 01010(2)

from_concat = 5'b01010 = 10(10), 01010(2)

narrow = 4'b0101 = 5(10), 0101(2)

trunc_then_shift = 4'b0010 = 2(10), 0010(2)

shift_then_trunc = 4'b1010 = -6(10), 1010(2)

②

because part-slect and concatenation will make signal be unsigned,

11.2

u > v ⇒ 2'b11 > 2'b01 ⇒ 3 > 1 ⇒ 1'b1

s > t ⇒ 2'b11 > 2'b01 ⇒ -1 > 1 ⇒ 1'b0

11.3

a	b	Low four sum bits	carry-out	signed overflow
0111	0001	1000	0	1
1111	0001	0000	1	0

NO, carry-out and signed overflow not the same, carry-out is the Fifth bit is 1 or not, signed overflow is the Low four sum bits sign is the same to a's sign (when a's sign and b's sign are the same) or not.

C-12 Design Flow and Concurrent Hardware

Verilog-2001/2005｜未指定 signed 的算術採 unsigned

Name: _____ Date: _____

C-12.1 Design-flow stages

Put RTL design and simulation, logic synthesis, and placement and routing in order. State what simulation, synthesis, and timing analysis check or produce.

C-12.2 ASIC versus FPGA

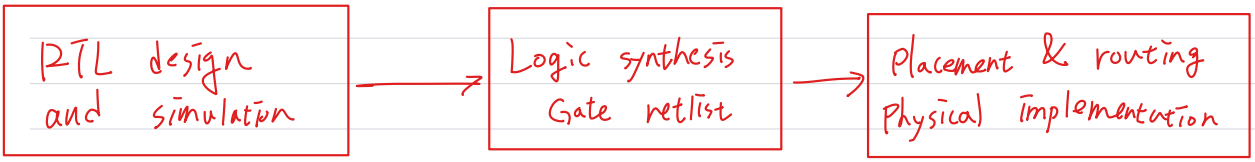
Give two implementation differences between an ASIC and an FPGA, following the review slides.

C-12.3 Elaboration and concurrency

Answer briefly: (a) Is a parameter a runtime input? (b) Do separate module instances and assign/always blocks execute in source-code order? (c) Does a procedural for loop by itself imply multiple clock cycles? (d) How many procedural writers should a register have in the coding style used in this course?

12.1

ANSWER / 作答區 請自行標示小題題號



Simulation checks behavior, Logic synthesis maps RTL to a gate netlist.
Placement and routing produce a physical implementation.
Timing analysis check path constraints.

12.2、12.3
略（當作我對、已作答完成）